

CS3211 Tutorial 1
AY 25/26 Sem 2 — github/omgeta

Q1. Producer–Producer:

- `jobs.push()` race with itself

Producer–Consumer:

- `jobs.push()` race with `jobs.empty()`
- `jobs.push()` race with `jobs.front()`
- `jobs.push()` race with `jobs.pop()`

Consumer–Consumer:

- `jobs.pop()` race with `jobs.empty()`
- `jobs.pop()` race with `jobs.front()`
- `jobs.pop()` race with itself

Q2. All accesses to `jobs` occur while holding `mut`. For a single mutex, `unlock()` synchronizes with a later successful `lock()` on that same mutex, and those lock/unlock operations are totally ordered. The queue accesses inside each critical section are sequenced with respect to the surrounding lock/unlock calls, so by transitivity they are ordered by happens-before across threads. Since a data race requires conflicting actions with neither happening-before the other, accesses to `jobs` are race-free.

Q3. `jobs.push()` throws an exception and `mut.lock()` is never called, so the lock is never released.